

MANUAL DEFINITIVO DE ENGENHARIA

Construindo Bots de Voz & Agentes de IA sem Alucinações

O guia de bastidores: por que IAs se perdem com áudio, como funciona o Function Calling real, técnicas anti-alucinação e a arquitetura à prova de falhas com Google Gemini.

Projeto: Bot de Alerta de Passagens Telegram + Google Flights

Tecnologias: Python, Google GenAI SDK, Telegram Bot API, libSQL / Turso Cloud

Versão: 4.0 (Arquitetura Desacoplada e Otimizada)

1. O Que Acontece por Baixo dos Panos com o Áudio

Para quem está de fora, mandar um áudio no Telegram parece simples: você grava a voz, o bot escuta e responde. Mas na engenharia de software, há uma cadeia complexa de conversões que pode quebrar facilmente se não for bem desenhada.

O Formato de Áudio do Telegram (OGG Opus)

Quando você solta o botão do microfone no Telegram, ele não envia um arquivo MP3 ou WAV comum. Ele gera um contêiner `.oga` / `.ogg` compactado com o codec **Opus**.

- **Vantagem:** O arquivo fica minúsculo (ideal para dados móveis).
- **Desafio:** É um fluxo comprimido de pacotes de frequência sonora. Não é texto, não possui pontuação e carrega ruídos de fundo, eco do ambiente e respiração.

⚠ O Erro Fatal que Acontecia (400 INVALID_ARGUMENT):

Inicialmente, tentamos pegar os bytes brutos desse áudio e enviar direto para a sessão de *Chat com Ferramentas (Function Calling)*.

Quando a IA ouvia o áudio e decidia chamar a ferramenta de busca no Google Flights, o histórico da conversa guardava: `[ÁUDIO BINÁRIO → CHAMADA DE FERRAMENTA → RESPOSTA DA FERRAMENTA]`. Na rodada seguinte, a API do Google tentava reenviar todo esse histórico contendo os bytes brutos do áudio anterior. A API não aceita reprocessar dados binários pesados de áudio dentro do histórico acumulado de ferramentas e abortava com **HTTP 400 Bad Request**.

A Arquitetura Vencedora: Transcrição Desacoplada

Para resolver isso definitivamente, separamos o **Ouvido** do **Cérebro de Ação** em duas etapas independentes:

- 1 **Recepção do Áudio:** O Telegram baixa o arquivo `.oga` para a memória temporária.
- 2 **Ouvido (Transcrição Isolada):** Uma chamada única e ultra-rápida (modelo Flash-Lite) apenas transcreve a fala humana em texto puro (ex: *"Quero ir de SP pro Rio dia 15/11 até 500 reais"*).
- 3 **Confirmação Visual:** O bot exibe na tela `Você disse: "..."`. O usuário sabe exatamente o que a máquina entendeu.
- 4 **Cérebro (TravelAgent):** O texto limpo é entregue ao Agente. Como é texto puro, o histórico nunca corrompe, o Function Calling funciona 100% e o banco de dados é atualizado sem erros.

2. Por Que a IA Precisa que Você "Descreva o Áudio"?

Uma dúvida muito comum de quem desenvolve com IA é: "Se o Gemini é um modelo multimodal, por que não posso apenas jogar o áudio lá dentro sem escrever nada?"

O Problema da Ambiguidade Multimodal

Para a rede neural, um arquivo de som é apenas uma matriz de frequências. Se você entrega um áudio sem contexto e sem comando, o modelo não sabe a sua **intenção de saída**:

- Ele deve *descrever os instrumentos musicais* que tocam ao fundo?
- Ele deve *resumir o sentimento da voz* (se a pessoa parece triste ou feliz)?
- Ele deve *responder diretamente como se fosse uma conversa por rádio*?
- Ou ele deve *transcrever estritamente a fala palavra por palavra*?

💡 A Regra de Ouro do Prompt de Transcrição:

Ao acompanhar o áudio com a instrução correta, você remove toda a margem de dúvida da rede neural:

"Você é um transcritor fiel em português. Transcreva exatamente as palavras faladas neste áudio. Se for apenas ruído ou silêncio, responda [SEM_FALA]. Retorne APENAS o texto falado, sem introduções ou explicações."

Isso força o modelo a desligar seu lado "poeta/conversador" e operar como um motor de Speech-to-Text de alta precisão.

3. Como Fazer a IA Não Alucinar (Zero Chutes)

Alucinação ocorre quando o modelo gera uma resposta que parece plausível e confiável, mas é factualmente falsa. Em sistemas de passagens aéreas, isso seria um desastre: o bot poderia inventar que existe um voo para Miami por R\$ 300 que não existe na vida real.

Os 4 Pilares Anti-Alucinação Aplicados no Projeto

1. Grounding Total (Ancoragem em Ferramentas Reais)

No prompt de sistema (`System Instruction`), a IA é formalmente proibida de inventar ou adivinhar preços. Ela é instruída:

```
"Você NÃO sabe os preços dos voos de cabeça. É estritamente PROIBIDO chutar valores. Sempre que o usuário quiser saber preços ou criar alertas, você DEVE obrigatoriamente executar a função buscar_voos_tempo_real para obter dados reais do Google Flights."
```

2. Temperatura Reduzida (0.1 a 0.3)

A temperatura regula o grau de aleatoriedade das previsões de palavras:

- **Temperatura 0.8 - 1.0 (Alta):** Ótima para escrever poesias, criar histórias de ficção e brainstorming. Péssima para lidar com dinheiro e rotas.
- **Temperatura 0.2 - 0.3 (Baixa - Nosso Padrão):** O modelo escolhe sempre as opções com maior probabilidade matemática de acerto, gerando respostas determinísticas, precisas e lógicas.

3. Function Calling Nativo com Validação de Tipos

Em vez de pedir para a IA retornar um texto livre com JSON (que pode vir quebrado), usamos as ferramentas nativas do SDK do Google:

- `origem: str` (resolvida para código IATA de 3 letras como BSB, GRU, GIG via `AirportService`).
- `teto: float` (garante que o valor financeiro seja um número e não texto).
- `data_ida: str` (validada pelo `DataService` para impedir datas no passado ou formatos americanos confusos).

4. Guardrails de Confirmação e Memória

Se o usuário diz apenas: "Quero ir para Salvador", o bot é instruído a **não inventar** data nem teto. Ele responde confirmando o que recebeu e pergunta exatamente o que falta:

"Ótimo! Salvador anotado. De onde você vai sair, em quais datas pretende viajar e qual o valor máximo que gostaria de pagar?"

4. Entendendo Cotas e Latência da API do Google

Durante o nosso desenvolvimento, nos deparamos com erros como `429 Too Many Requests` e `504 Gateway Timeout` . Entender por que eles acontecem é o segredo de qualquer desenvolvedor sênior.

Modelo	Cota Free Tier	Tempo Resposta Médio	Uso Recomendado
gemini-3.5-flash	20 req / dia (rígido)	~2.5 segundos	Apenas para testes pontuais sem automação.
gemini-3.7-flash	Variável	30s a 100s (com Thinking)	Raciocínio complexo, matemática e código. Muito lento para bots de chat.
gemini-flash-lite-latest	1.500 req / dia	0.7s a 1.5s	Campeão absoluto para Bots Telegram, transcrição e suporte em tempo real.

 A Pegadinha do Thinking Mode (Gemini 3.7 Flash):

O Gemini 3.7 foi lançado como um modelo híbrido com modo de "pensamento profundo" (*thinking mode*) ativado por padrão. Ele gasta dezenas de segundos simulando linhas de raciocínio antes de soltar a primeira palavra.

Para bots de mensagens instantâneas onde as pessoas esperam respostas em 2 segundos, isso causa a impressão de que o bot "travou" ou tomou timeout no Telegram.

 A Pegadinha do Timeout (Milissegundos vs Segundos):

Na SDK `google-genai` , o parâmetro `HttpOptions(timeout=...)` espera o tempo em **milissegundos**. Se você colocar `timeout=15.0` pensando em 15 segundos, o código divide por 1000 e cria um timeout de **0,015 segundos** (15 milissegundos), abortando todas as conexões na largada! O correto para 30 segundos é `timeout=30000` .

5. O Que Nunca Depende de Cota de IA (Custo Zero e Infinito)

A melhor arquitetura de software é aquela que usa a IA apenas onde ela brilha (compreensão de linguagem e flexibilidade humana), mas mantém a espinha dorsal em código tradicional robusto:

- **Scraping de Voos (fast-flights):** Consulta o Google Flights diretamente, trazendo preços reais em Reais (BRL), sem pagar US\$ 0,02 por consulta em APIs aéreas corporativas (como Amadeus ou Skyscanner).
- **Verificação em Background (Scheduler):** Roda de 3 em 3 horas no Heroku varrendo todos os alertas do banco sem gastar 1 centavo de IA.
- **Banco na Nuvem (Turso libSQL):** Armazena os usuários, permissões e histórico de preços com persistência permanente e segura fora do filesystem do Heroku.
- **Assistente Passo a Passo (/novo):** Interface baseada em botões e perguntas estruturadas. Funciona até se a internet da OpenAI ou do Google cair no mundo todo.

6. Resumo Executivo: Checklist para Seus Próximos Bots

1. **Nunca mande bytes de áudio para dentro do chat multi-turn:** Transcreva o áudio primeiro de forma atômica; envie apenas o texto transcrito para o agente.
2. **Escolha o modelo certo para a função certa:** Modelos *Flash-Lite* para tarefas em tempo real de alta volumetria (1.500 req/dia); modelos pesados apenas quando houver raciocínio matemático denso.
3. **Grounding Obrigatório:** Se o seu bot precisa de fatos (preços, estoque, agendamentos), force-o a usar ferramentas e proíba qualquer resposta inventada no prompt.
4. **Sempre forneça uma alternativa sem IA:** Usuários em conexões lentas ou com comandos rotineiros preferem botões de 1 clique (`/novo` , [🔄 Checar Agora]) em vez de conversar longamente.